Aprendizaje Profundo Generativo

Redes Bayesianas Profundas/ Modelos Autoregresivos

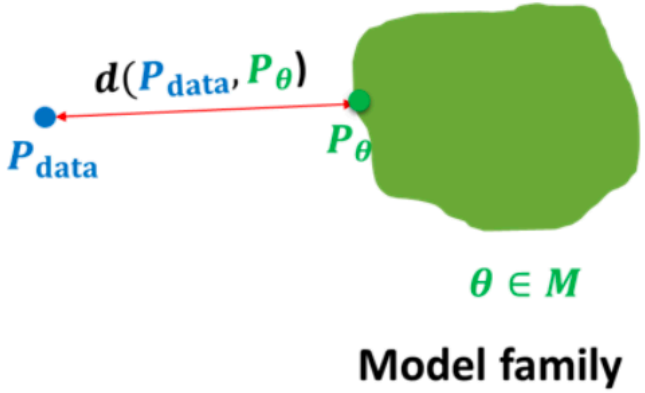
Pablo Martínez Olmos, pamartin@ing.uc3m.es
Alejandro Lancho Serrano, alancho@ing.uc3m.es
Vanessa Gómez Verdejo, vanessag@ing.uc3m.es

Modelos Generativos

- Conjunto de entrenamiento de ejemplos, por ejemplo, imágenes de perros.



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



- Nuestro objetivo es aprender una distribución de probabilidad $p(x)$ tal que …

1. **Generación:** Si muestreamos $x_{\text{new}} \sim p(x)$ debe parecer un perro

2. **Estimation de densidad**: $p(x)$ debe tener un valor alto si la imagen parece un perro, y bajo por el contrario (detección de anomalías)

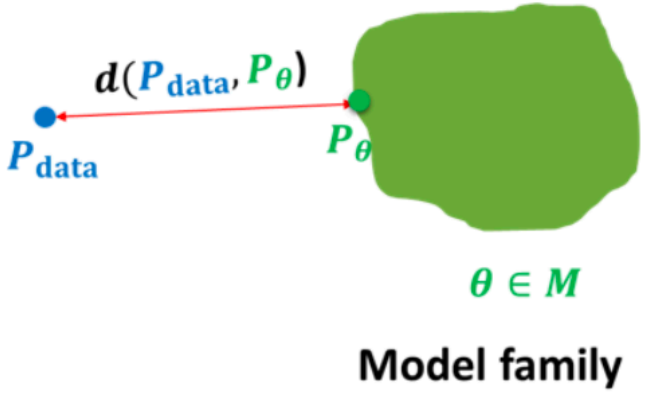
3. **Representación no supervisada**: poder encontrar de forma automática qué tienen en común las imágenes (orejas, rabo, hocico …)

Modelos Generativos

- Conjunto de entrenamiento de ejemplos, por ejemplo, imágenes de perros.

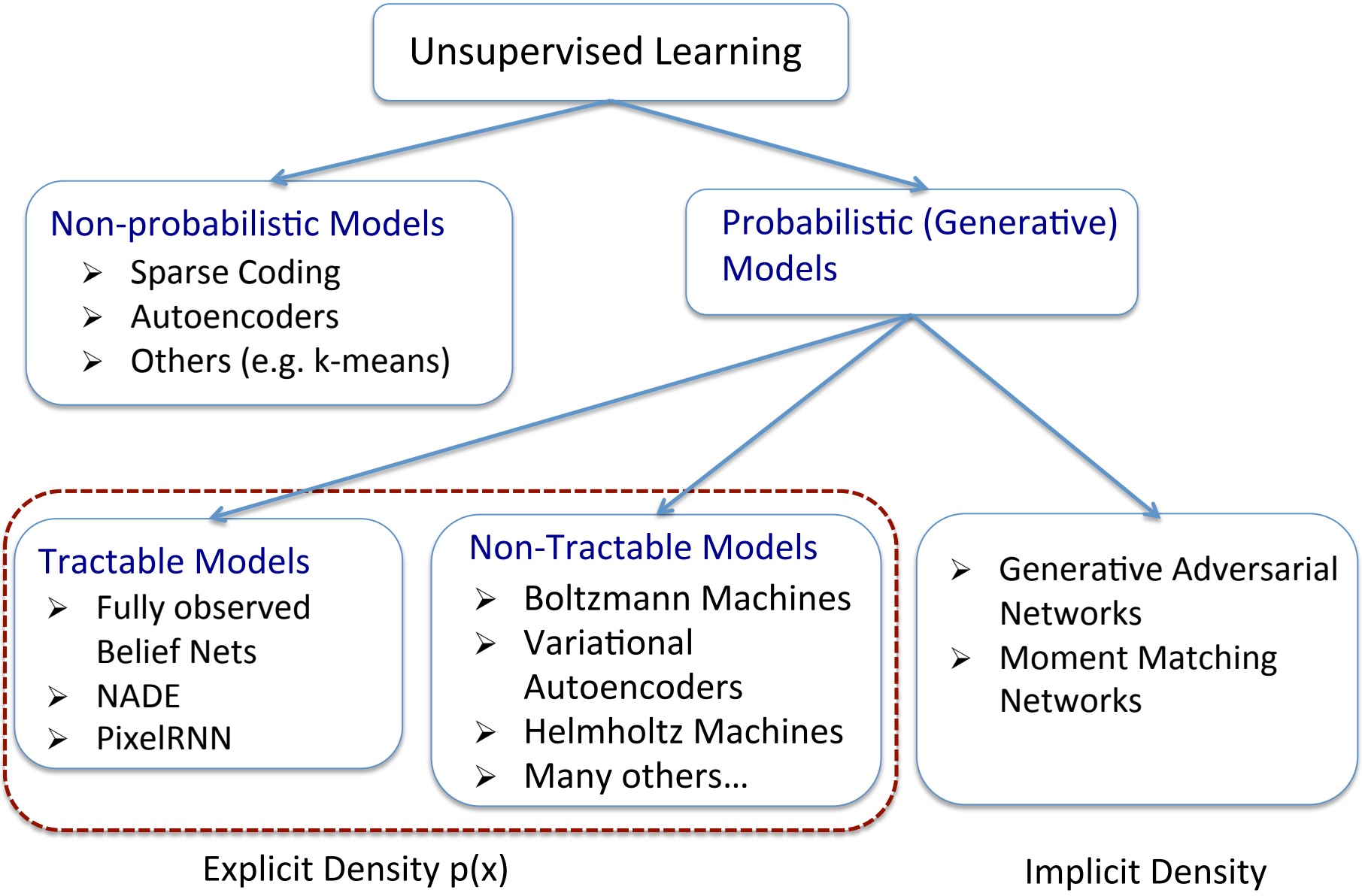


$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



- **Paso 1:** Cómo represento (parameterizo) $p(x)$?

- **Paso 2:** Cómo aprendo?


Unsupervised Learning
Non-probabilistic Models
Sparse Coding
Autoencoders
Others (e.g. k-means)
Probabilistic (Generative) Models
Tractable Models
Fully observed Belief Nets
NADE
PixelRNN
Non-Tractable Models
Boltzmann Machines
Variational Autoencoders
Helmholtz Machines
Many others…
Generative Adversarial Networks
Moment Matching Networks
Explicit Density p(x)
Implicit Density

Distribuciones de probabilidad, un repaso rápido

- Dos reglas importantes que debemos recordar

Marginalización: $p(x) = \sum_{y} p(x, y)$

Factorización: $p(x, y) = p(y \mid x)p(x) = p(x \mid y)p(y)$ ← Teorema de Bayes!

- Usando la regla de factorización, podemos represéntatela **cualquier** distribución

$$p(\mathbf{x}) = p(x_1)\prod_{d=2}^{D} p(x_d \mid \mathbf{x}_{<d})$$

donde $\mathbf{x}_{<d} = [x_1, \ldots, x_{d-1}]^T$

$$p\left(x_1, x_2, x_3, x_4\right) =$$
$$p\left(x_1\right)p\left(x_2 \mid x_1\right)p\left(x_3 \mid x_1, x_2\right)p\left(x_4 \mid x_1, x_2, x_3\right)$$

Deep belief networks (a.k.a. deep autoregressive models)

- Modelamos explícitamente todas las densidades condicionales

$$p_{\text{model}}(\boldsymbol{x}) = p_{\text{model}}(x_1) \prod_{i=2}^{n} p_{\text{model}}(x_i | x_1, \ldots, x_{i-1})$$

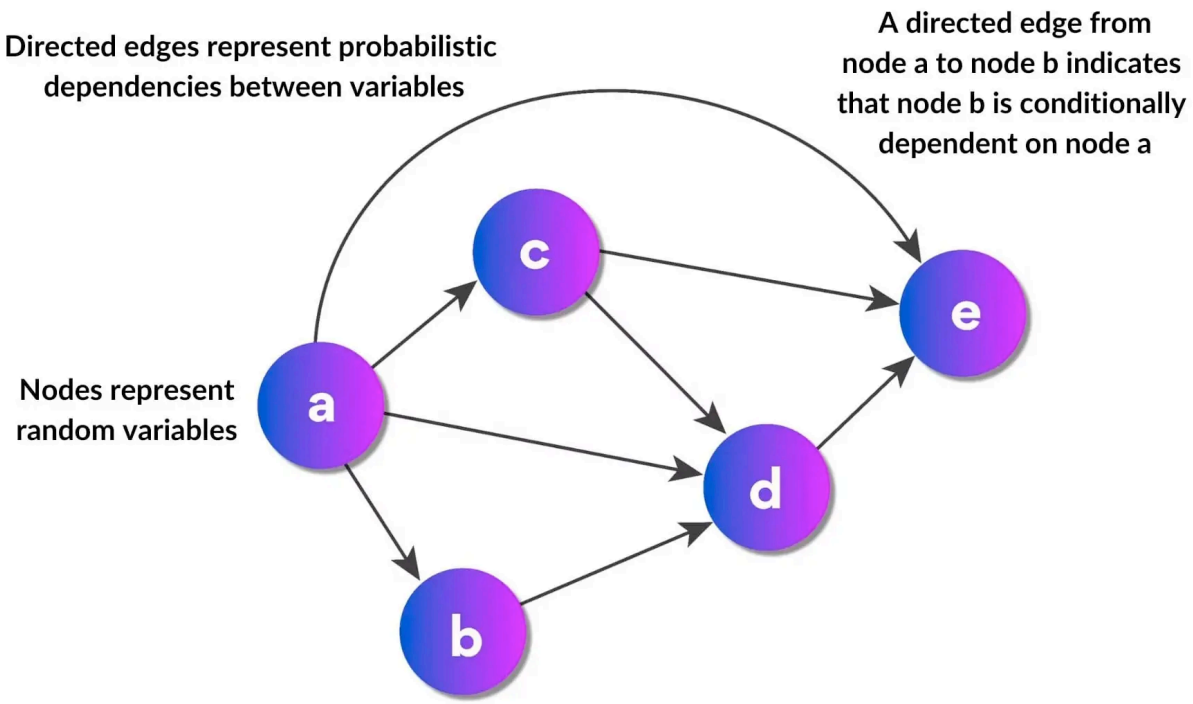
Cada factor puede estar modelado por una red neuronal distinta

- Entrenamiento por máxima log-verosimilitud

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \log p_{\text{model}}(\mathbf{x}_n)$$

Modelar todos los términos por separado puede ser complejo …

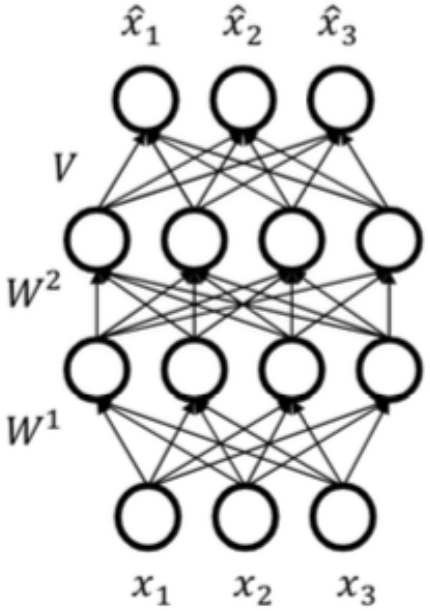
Deep belief networks



Autoregressive Autoencoders

- Utilizamos una única red neuronal para generar las distribuciones condicionales
- Partimos de la idea de un *autoencoder* determinista e introducimos modificaciones
- Supongamos tres dimensiones (D=3) y que el orden es

$$p\left(x_1, x_2, x_3\right) = p\left(x_2\right) p\left(x_3 \mid x_2\right) p\left(x_1 \mid x_2, x_3\right)$$



Autoencoder

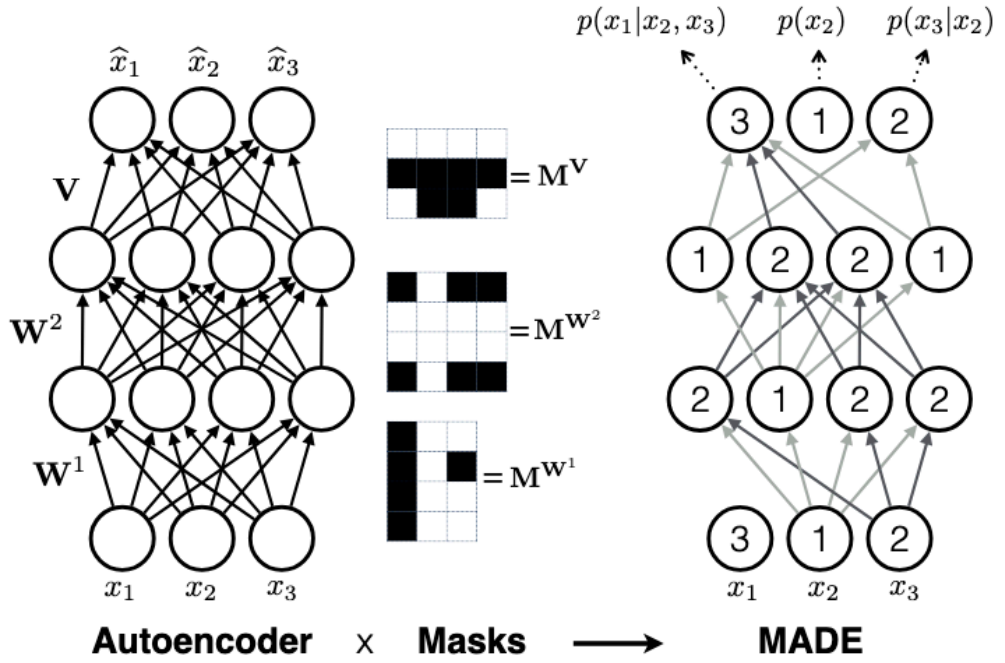
- $\hat{x}_2$ no puede depender de ninguna entrada
- $\hat{x}_3$ sólo puede depender de $\hat{x}_2$
- …

Directed Acyclic Graph (DAG)

- La salida debe parameterizar la distribución condicional, generando sus momentos

MADE: Masked autoencoders for Distribution Estimation

- Imponemos una estructura DAG **etiquetando los nodos** y usando **máscaras**.

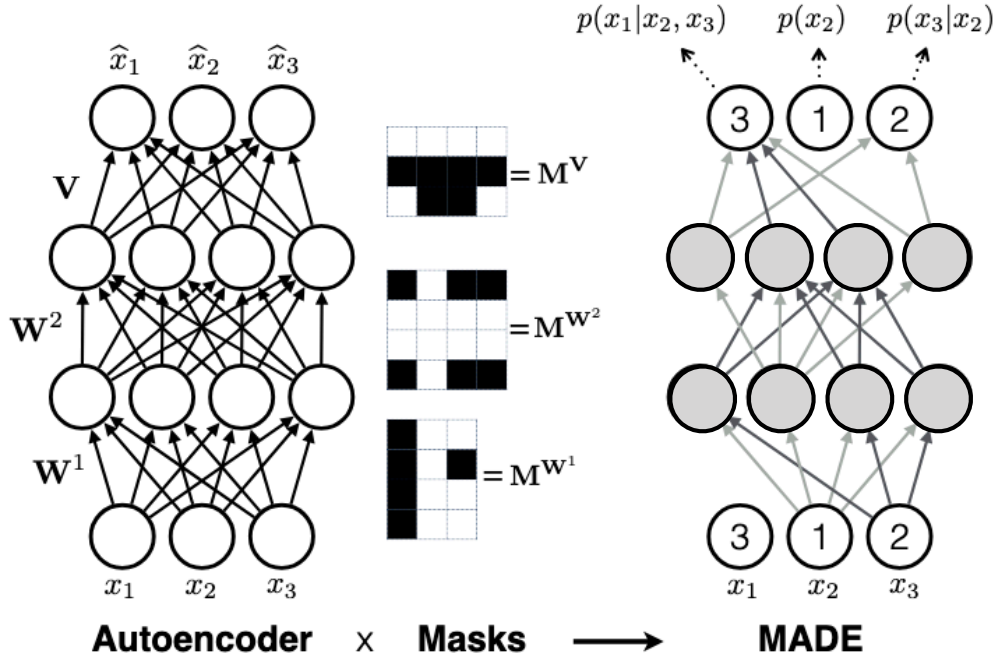


$$p\left(x_1, x_2, x_3\right) = p\left(x_2\right) p\left(x_3 \mid x_2\right) p\left(x_1 \mid x_2, x_3\right)$$

MADE: Masked autoencoders for Distribution Estimation

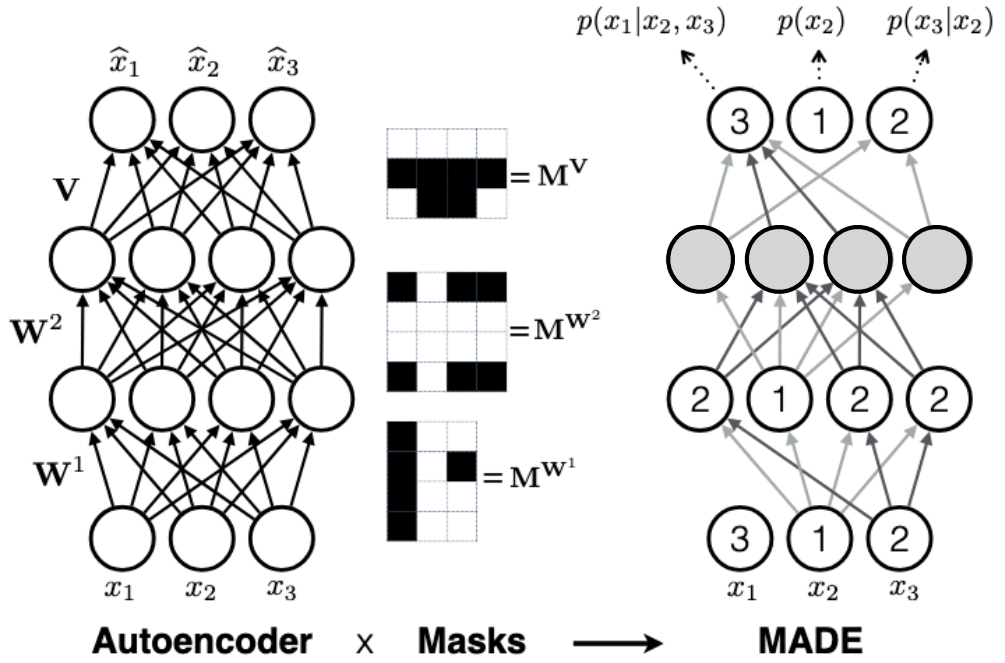
1. A la entrada y salida, mantenemos el orden establecido

$$p\left(x_1, x_2, x_3\right) = p\left(x_2\right) p\left(x_3 \mid x_2\right) p\left(x_1 \mid x_2, x_3\right)$$



MADE: Masked autoencoders for Distribution Estimation

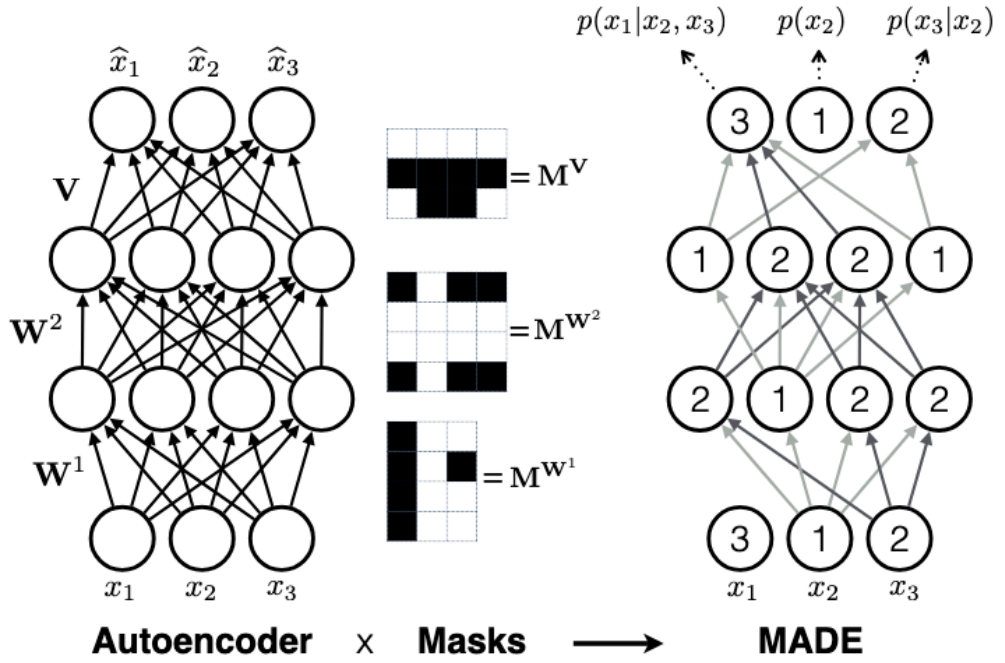
2. A cada unidad de cada capa, le asignamos un número aleatorio entre 1 y D-1



$$p(x_1, x_2, x_3) = p(x_2)p(x_3 \mid x_2)p(x_1 \mid x_2, x_3)$$

MADE: Masked autoencoders for Distribution Estimation

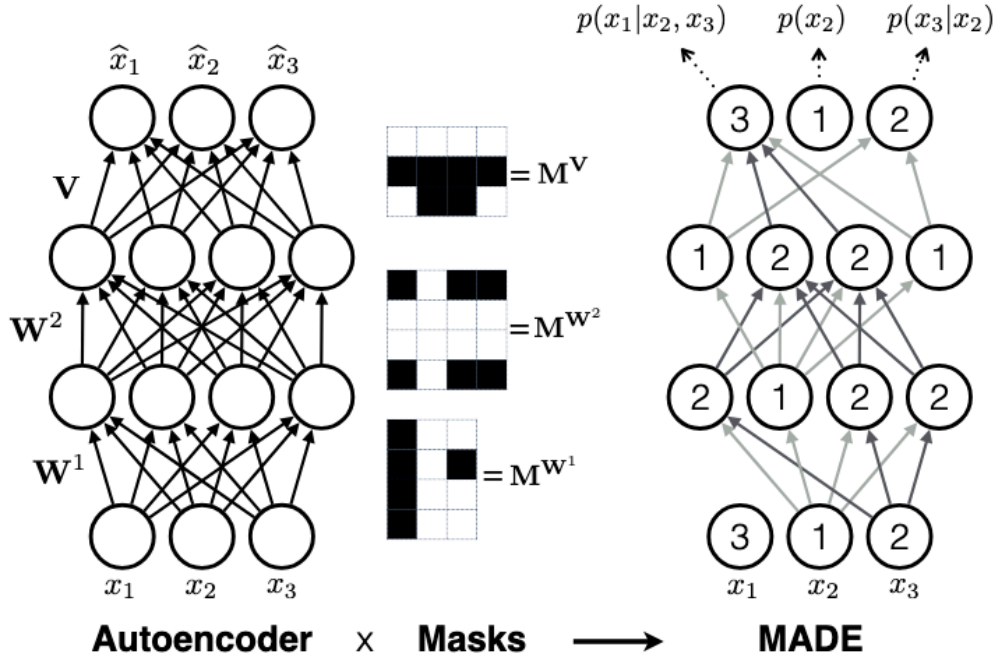
2. A cada unidad de cada capa, le asignamos un número aleatorio entre 1 y D-1



$$p(x_1, x_2, x_3) = p(x_2)p(x_3 \mid x_2)p(x_1 \mid x_2, x_3)$$

MADE: Masked autoencoders for Distribution Estimation

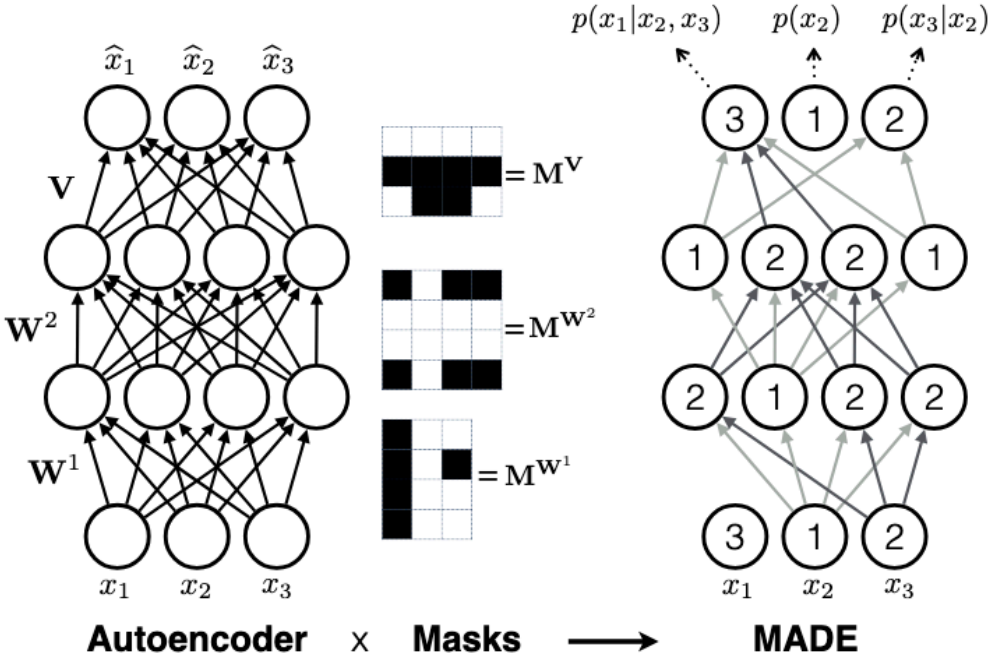
3. Conectamos cada unidad de la última capa a las unidades de la capa anterior con etiqueta menor. En las capas anteriores, con etiqueta menor o igual



$$p\left(x_1, x_2, x_3\right) = p\left(x_2\right) p\left(x_3 \mid x_2\right) p\left(x_1 \mid x_2, x_3\right)$$

MADE: Masked autoencoders for Distribution Estimation

4. Parameterizamos cada distribución condicional de acuerdo a la naturaleza de los datos.

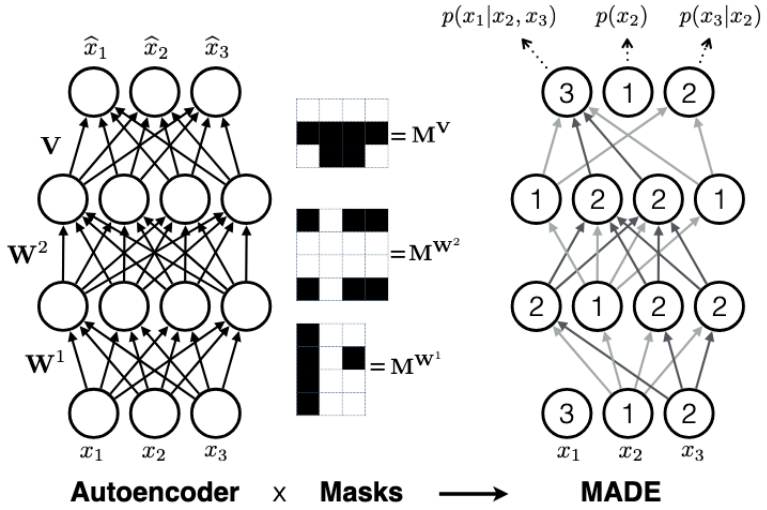


$$p(x_1|x_2,x_3) = \mathcal{N}(\hat{x}_1, 1)$$
$$p(x_3|x_2) = \text{Ber}(\sigma(\hat{x}_3))$$

$$p\left(x_1, x_2, x_3\right) = p\left(x_2\right) p\left(x_3 \mid x_2\right) p\left(x_1 \mid x_2, x_3\right)$$

MADE: Masked autoencoders for Distribution Estimation

5. Entrenamiento por máxima verosimilitud



$$p_{\text{model}}(\boldsymbol{x}) = p_{\text{model}}(x_1) \prod_{i=2}^{n} p_{\text{model}}(x_i | x_1, \ldots, x_{i-1})$$

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \log p_{\text{model}}(\mathbf{x}_n)$$

Order-agnostic training

Entrenar con ordenamientos aleatorios mejora la imputación y permite construir ensembles.

- **Marginalización eficiente**: ordenar primero las variables observadas facilita la inferencia.
- **Ensembles** en tiempo real: diferentes ordenamientos generan modelos ligeramente distintos; promediarlos mejora el rendimiento.

Ejemplo MADE

MADE: Masked Autoencoder for Distribution Estimation

Curso Inteligencia Artificial Generativa

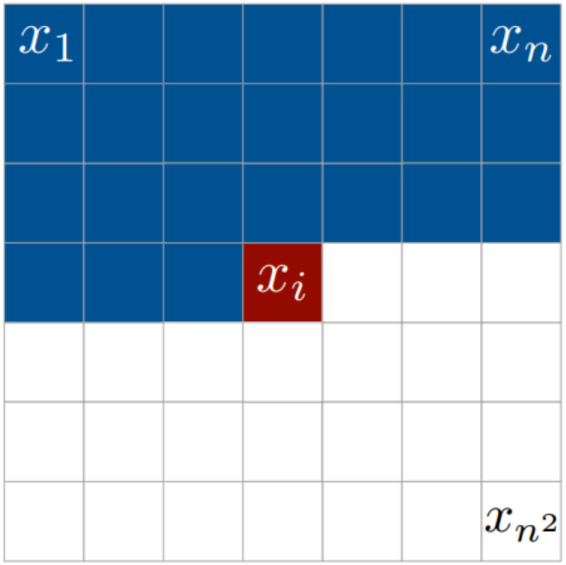
Pablo Martínez Olmos, Vanessa Gómez Verdejo, Alejandro Lancho Serrano

Departamento de Teoría de la Señal y Comunicaciones

Universidad Carlos III de Madrid

uc3m | Universidad Carlos III de Madrid

MADE is a probabilistic autoregressive model designed to exploit potential dependencies among the components of an input vector X. It builds a separate predictive model for each eleme autoregression—where elements are predicted based on others within the same input. MADE implements this through strategically masking certain connections in the neural network's w that each variable is only conditioned on a defined subset of others.



Order-Agnostic Autoregressive Models (OA-ARMs)

A Deep and Tractable Density Estimator

Benigno Uria B.URIA@ED.AC.UK
Iain Murray I.MURRAY@ED.AC.UK
School of Informatics, University of Edinburgh

Hugo Larochelle HUGO.LAROCHELLE@USHERBROOKE.CA
Département d'informatique, Université de Sherbrooke

ICML 2014

XLNet: Generalized Autoregressive Pretraining for Language Understanding

Zhilin Yang[*1], **Zihang Dai**[*12], **Yiming Yang**[1], **Jaime Carbonell**[1], **Ruslan Salakhutdinov**[1], **Quoc V. Le**[2]
[1]Carnegie Mellon University, [2]Google AI Brain Team
{zhiliny,dzihang,yiming,jgc,rsalakhu}@cs.cmu.edu, qvl@google.com

NeurIPS 2019

AUTOREGRESSIVE DIFFUSION MODELS

Emiel Hoogeboom[*], **Alexey A. Gritsenko, Jasmijn Bastings, Ben Poole, Rianne van den Berg, Tim Salimans**
Google Research
e.hoogeboom@uva.nl, {agritsenko,pooleb,bastings,salimans}@google.com, riannevdberg@gmail.com

ICLR 2022

Order-Agnostic Autoregressive Difussion Models (OA-ARDM)

- OA-ARMs generate variables with a **random order** $\sigma \in \mathcal{S}_D$

Latent Variable!

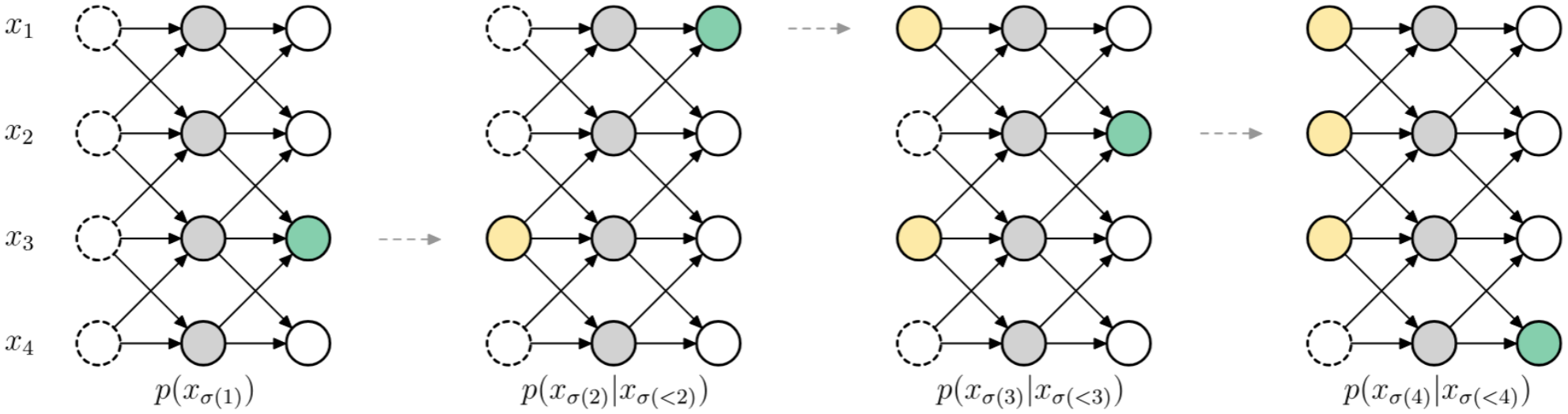


Figure 1: Generation of Autoregressive Diffusion Models for the generation order $\sigma = (3, 1, 2, 4)$. Filled circles in the first and third layers represent respectively the input and output variables, and the middle layer represents internal activations of the network.

Order-Agnostic Autoregressive Difussion Models (OA-ARDM)

- OA-ARMs generate variables with a **random order** $\sigma \in \mathcal{S}_D$

Latent Variable!

$$\log p(x) = \mathbb{E}_{\sigma \sim \mathcal{U}(S_D)} \log p(\mathbf{x}|\sigma) \geq \mathbb{E}_{\sigma \sim \mathcal{U}(S_D)} \sum_{t=1}^{D} \log p(x_{\sigma(t)} \mid x_{\sigma(<t)})$$

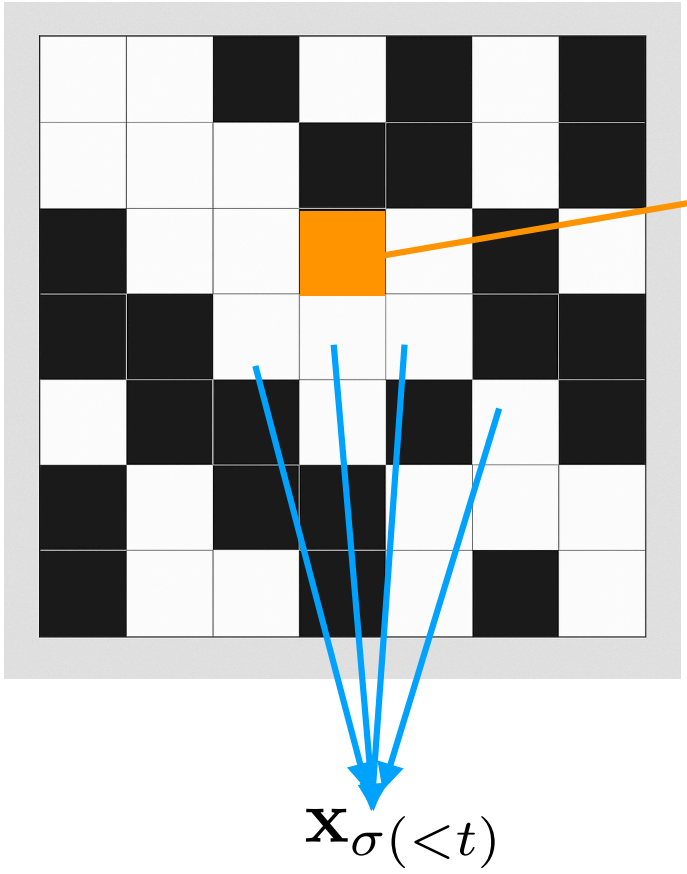
$$\sum_{t=1}^{D} a_t = D \cdot \mathbb{E}_{t \sim \mathcal{U}(1,\ldots,D)}[a_t]$$



$$\log p(x) \geq D \cdot \mathbb{E}_{t \sim \mathcal{U}(1,\ldots,D)} \mathbb{E}_{\sigma \sim \mathcal{U}(S_D)} \log p(x_{\sigma(t)} \mid x_{\sigma(<t)})$$

Order-Agnostic Autoregressive Difussion Models (OA-ARDM)

$$\log p(x) \geq D \cdot \mathbb{E}_{t \sim \mathcal{U}(1,\ldots,D)} \mathbb{E}_{\sigma \sim \mathcal{U}(S_D)} \log p(x_{\sigma(t)} \mid x_{\sigma(<t)})$$



Why not averaging the objective across **all unobserved dimensions at time t**?

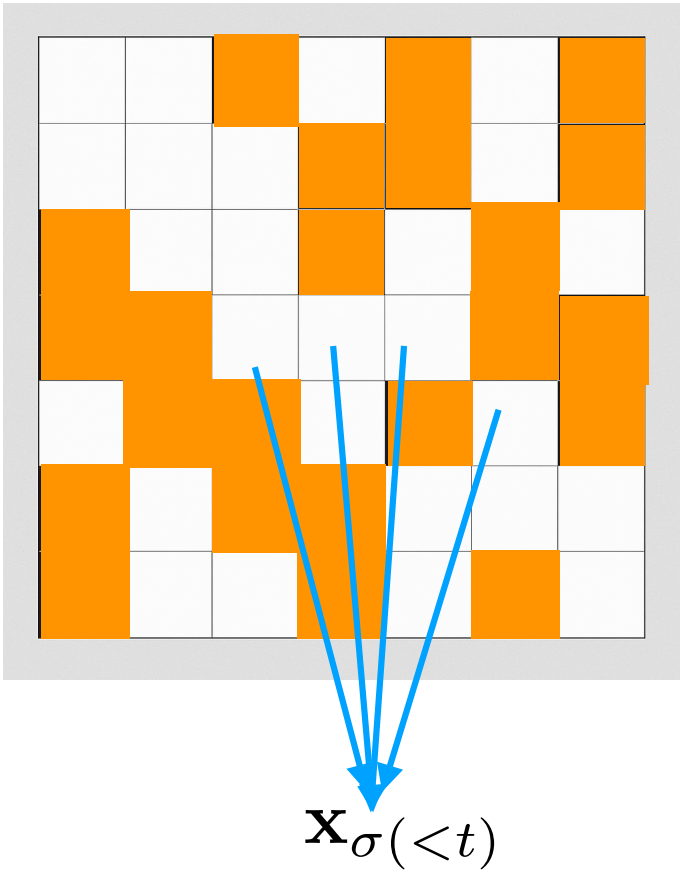
We can marginalize all possible permutations from time t

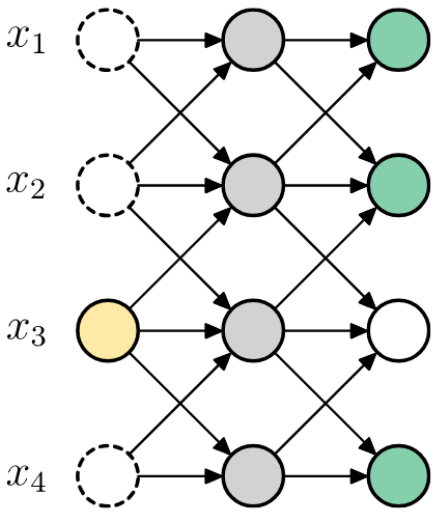
Order-Agnostic Autoregressive Difussion Models (OA-ARDM)

- At time *t* there are exactly *D-t+1* masked variables. We predict them all

$$\log p(x) \geq \mathbb{E}_{t \sim U(1,\ldots,D)}[D \cdot L_t]$$

$$L_t = \frac{1}{D-t+1} \mathbb{E}_{\sigma \sim U(S_D)} \sum_{k \in \sigma(\geq t)} \log p(x_k \mid x_{\sigma(<t)})$$





$$\sigma(1) = 3$$

A **single neural network** is utilized and shared for different σ and t, using a **binary mask** at each t.

Order-Agnostic Autoregressive Difussion Models (OA-ARDM)

Algorithm 1 Sampling from OA-ARDMs

Input: Network f
Output: Sample $\boldsymbol{x}$
Initialize $\boldsymbol{x} = \mathbf{0}$
Sample $\sigma \sim \mathcal{U}(S_D)$
for t in $\{1, \dots, D\}$ **do**
  $\boldsymbol{m} \leftarrow (\sigma < t)$ and $\boldsymbol{n} \leftarrow (\sigma = t)$
  $\boldsymbol{x}' \sim \mathcal{C}(\boldsymbol{x} | f(\boldsymbol{m} \odot \boldsymbol{x}))$
  $\boldsymbol{x} \leftarrow (1 - \boldsymbol{n}) \odot \boldsymbol{x} + \boldsymbol{n} \odot \boldsymbol{x}'$

Algorithm 2 Optimizing OA-ARDMs

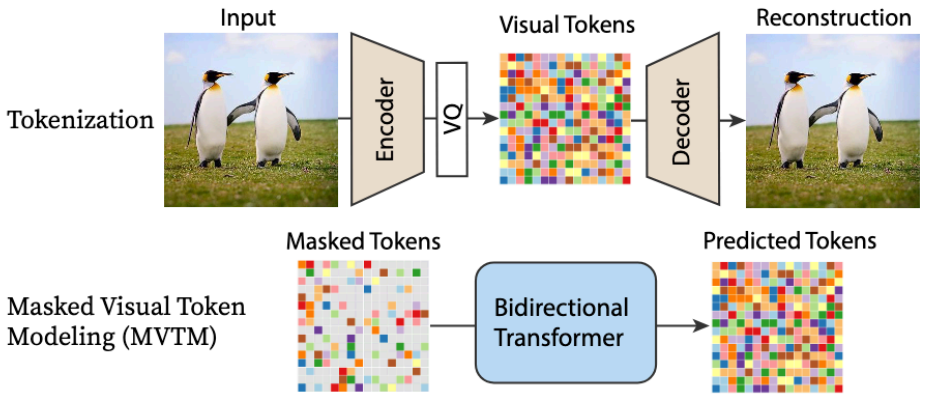
Input: Datapoint $\boldsymbol{x}$, Network f
Output: ELBO $\mathcal{L}$
Sample $t \sim \mathcal{U}(1, \dots, D)$
Sample $\sigma \sim \mathcal{U}(S_D)$
Compute $\boldsymbol{m} \leftarrow (\sigma < t)$
$\boldsymbol{l} \leftarrow (1 - \boldsymbol{m}) \odot \log \mathcal{C}(\boldsymbol{x} | f(\boldsymbol{m} \odot \boldsymbol{x}))$
$\mathcal{L}_t \leftarrow \frac{1}{D-t+1} \text{sum}(\boldsymbol{l})$
$\mathcal{L} \leftarrow D \cdot \mathcal{L}_t$



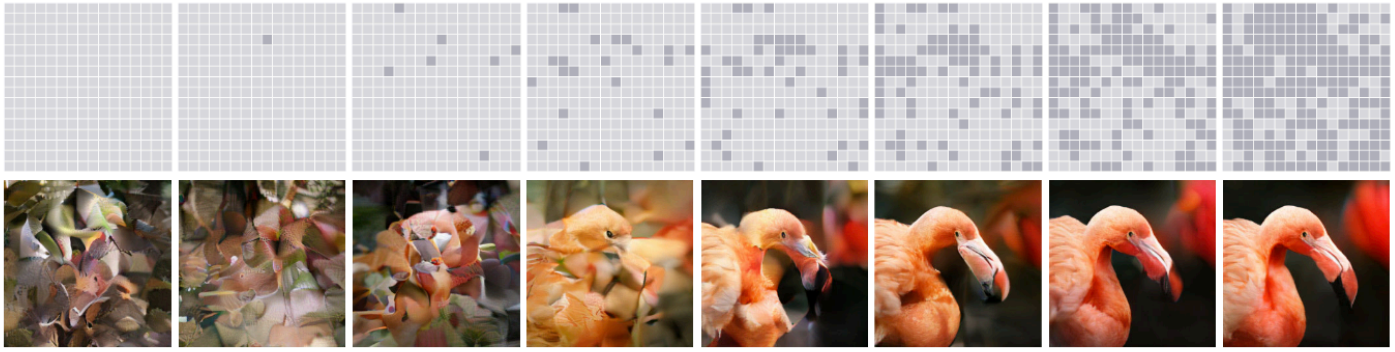
Order-Agnostic Autoregressive Difussion Models (OA-ARDM)

MaskGIT: Masked Generative Image Transformer

Huiwen Chang Han Zhang Lu Jiang Ce Liu* William T. Freeman
Google Research



Scheduled Parallel Decoding with MaskGIT

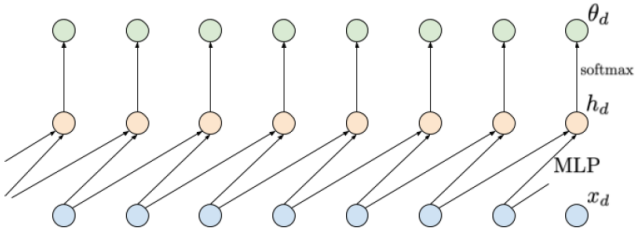


Modelos Autoregresivos para Series Temporales

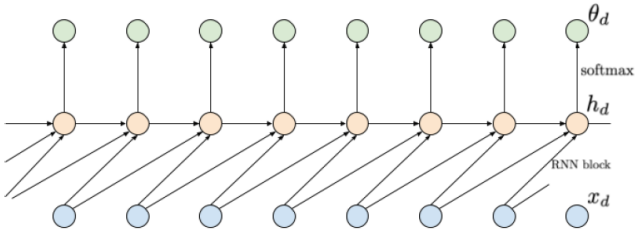
Modelos autoregresivos para series temporales

- Parameterizamos los términos de la distribución conjunta usando redes secuenciales

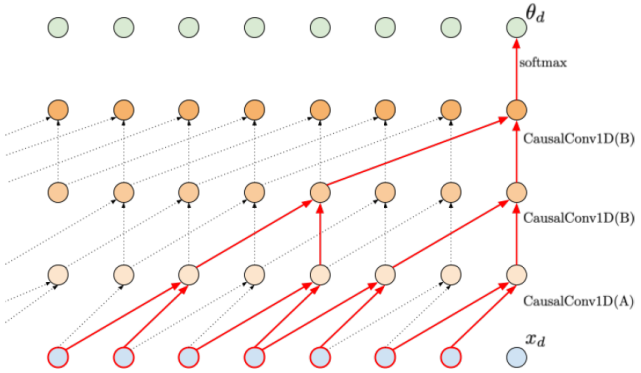
$$p_{\text{model}}(\boldsymbol{x}) = p_{\text{model}}(x_1) \prod_{i=2}^{n} p_{\text{model}}(x_i | x_1, \ldots, x_{i-1})$$



Memoria finita: MLPs



Memoria arbitraria (RNNs)

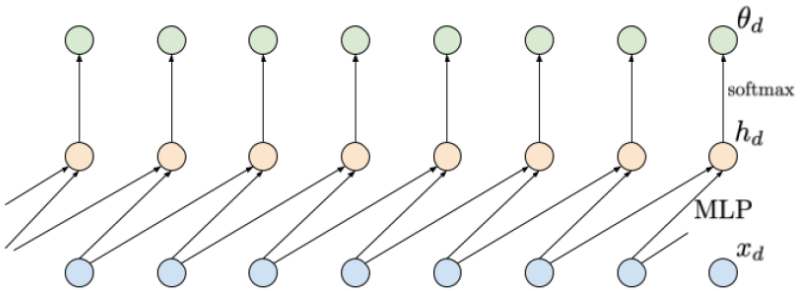


Memoria arbitraria (CNNs)

Estado del arte: **Transformers**!

Parameterización con memoria finita

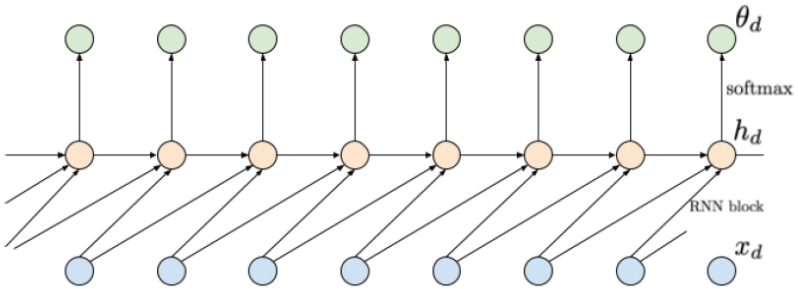
- Por ejemplo, fijarnos en las dos variables anteriores para predecir la siguiente
- Una única red a lo largo del tiempo

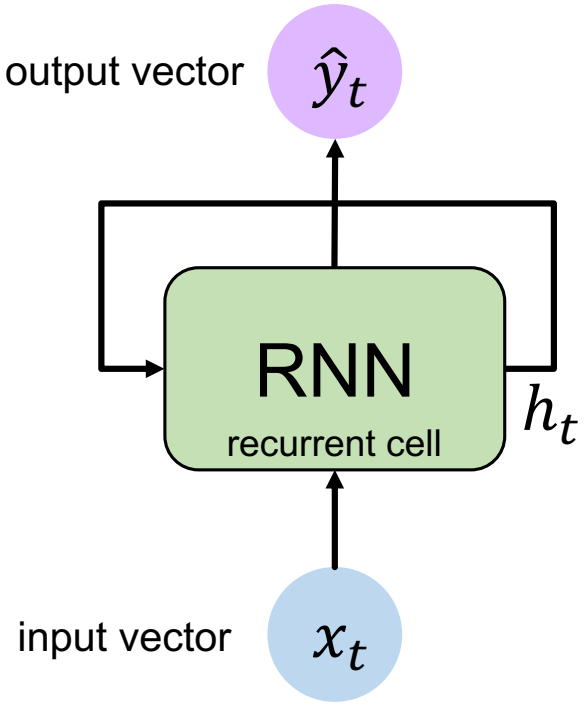


$$p(x_d \mid x_{d-1}, x_{d-2}) = \text{Cat}(\text{softmax}(\text{NN}(x_{d-1}, x_{d-2})))$$

Eficiente, pero es una limitación importante

Memoria arbitraria: RNNs





Por ejemplo:

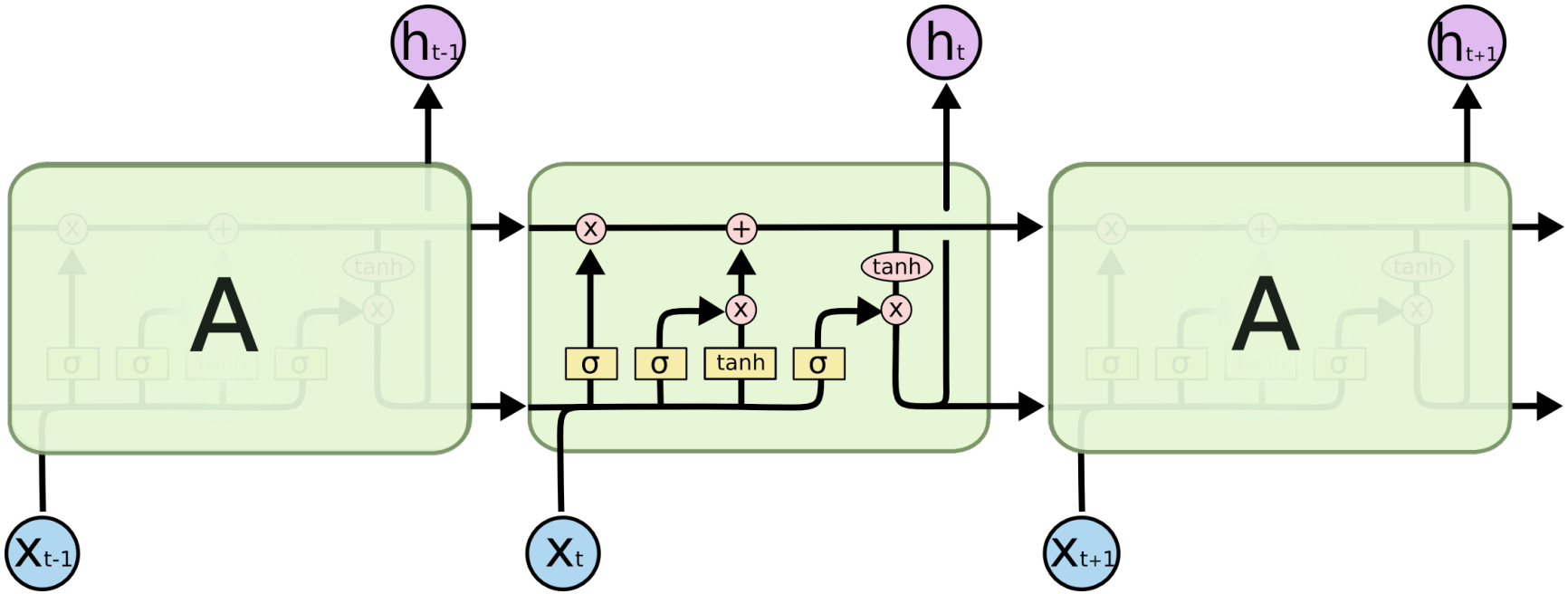
$$p(x_d \mid x_{d-1}, x_{d-2}) = \text{Cat}(\text{softmax}(\text{RNN}(x_{d-1}, h_{d-1})))$$

No hay una parameterización explicita de la memoria

Exploding/vanishing gradients, no son paralelizables

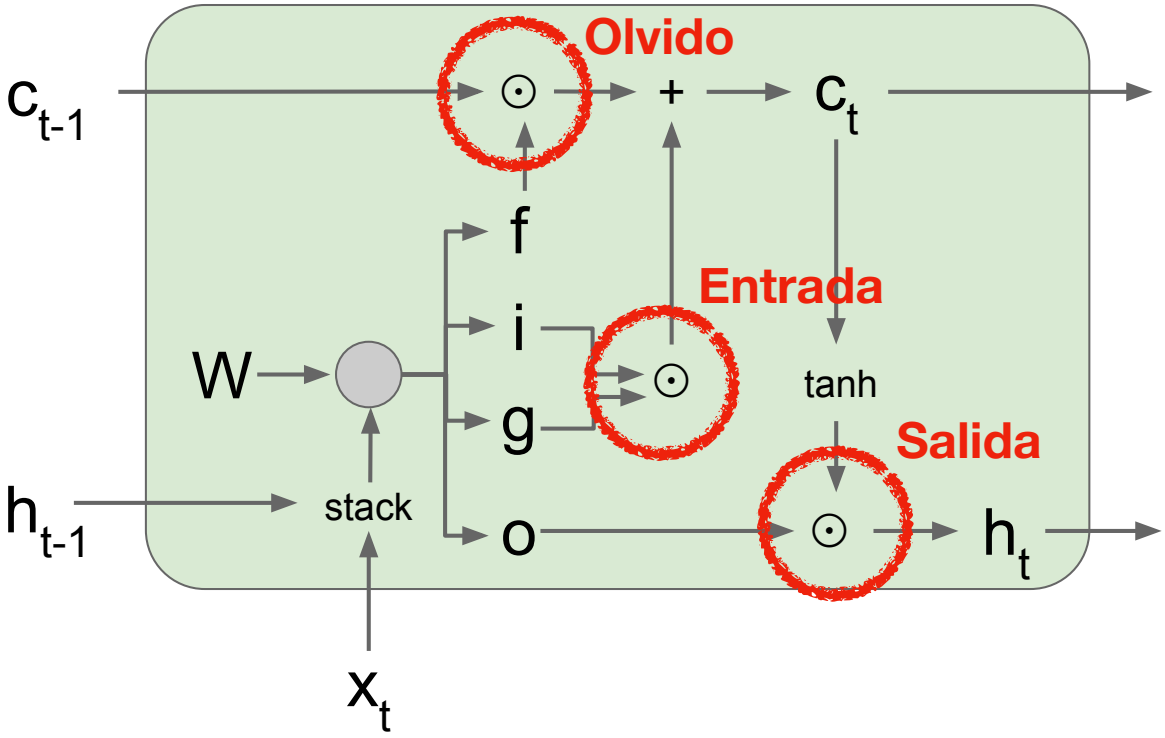
Redes recurrentes con “puertas”: gated cells

Unidad recurrente **más compleja con puertas** para controlar qué información se transmite



© MIT 6.S191: Introduction to Deep Learning

Mecanismo de puertas en LSTMs



$$f = \sigma\left(W_{fx}x_t + W_{fh}h_{t-1} + b_f\right)$$
$$i = \sigma\left(W_{xi}x_t + W_{hi}h_{t-1} + b_i\right)$$
$$o = \sigma\left(W_{ox}x_t + W_{oh}h_{t-1} + b_o\right)$$
$$g = \tanh\left(W_{cx}x_t + W_{ch}h_{t-1} + b_c\right)$$
$$c_t = f \circ c_{t-1} + i \circ \tilde{c}_t$$
$$h_t = o_t \circ \tanh\left(c_t\right)$$

Las puertas filtran la información que dejan pasar mediante una **salida sigmoide**

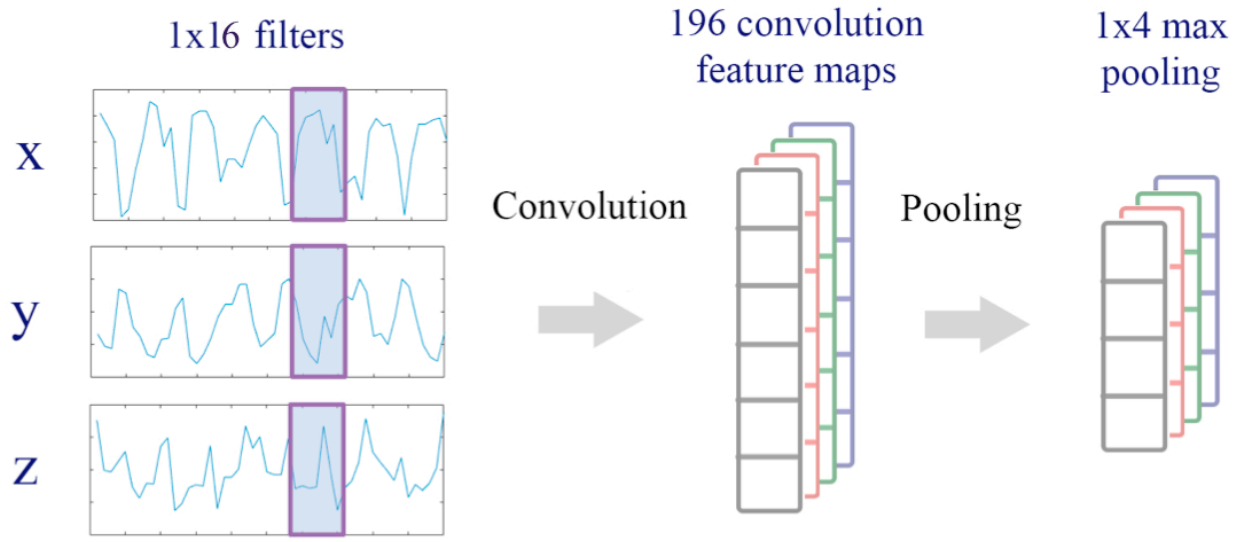
y

multiplicación elemento a elemento $\odot$

@Stanford CS231
http://cs231n.stanford.edu

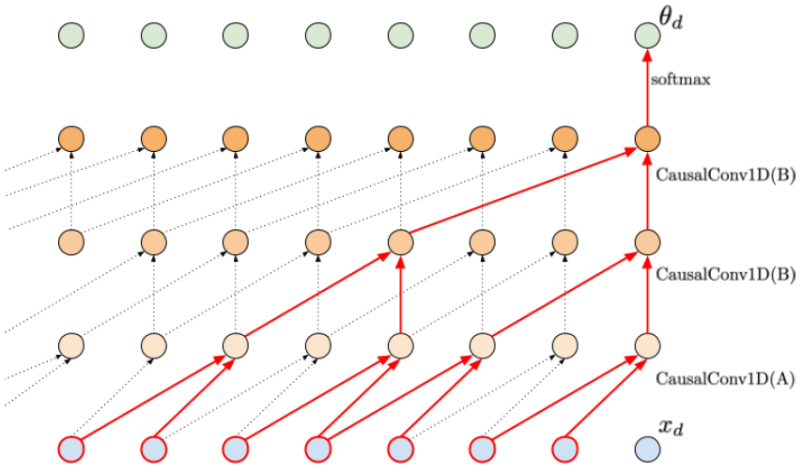
1D CNNs

- De forma similar para con imágenes, podemos aprender **filtros temporales con CNNs.**
- Alternativa a RNNs, mucho más **rápida de entrenar y evaluar**.
- Típicamente RNNs y 1D-CNNs **se usan de forma conjunta**. Con la red convolucional capturamos correlaciones locales y con la RNN correlaciones más profundas.



CNNs

- Alternativamente podemos utilizar redes convolucionales de tipo 1D



$$p(x_d \mid x_{d-1}, x_{d-2}) = \text{Cat}(\text{softmax}(\text{CNN}(\mathbf{x}_{<d})))$$

Additional tip: Utilizar ***dilation*** para mejorar las dependencias a largo plazo

Entrenamiento más estable y eficiente

Peores prestaciones en términos de memoria a medio-largo plazo que las RNNs

Ejemplo de RNNs para predicción de series temporales

DeepAR with RNNs and Transformers for Time Series Forecasting

Generative Deep Learning Course

Pablo Martínez Olmos, Vanessa Gómez Verdejo, Emilio Parrado Hernández

Department of Signal Theory and Communications

uc3m | Universidad Carlos III de Madrid

In this example, we will develop a simple probabilistic model for time series, in which an RNN will predict the expected evolution of a time series. Given a set of signals $\mathcal{D} = \{X^1, X^2, \dots, X^N\}$ where

$$X^i = [X_0^i, X_1^i, \dots, X_T^i]$$

is the i-th signal, we will fit a probabilistic model of the form

$$p(X|X_0) = \prod_{t=1}^{T} p(X_t|\mathbf{X}_{0:t-1})$$

where $\mathbf{X}_{0:t-1} = [X_0, X_1, \dots, X_{t-1}]$ and each conditional factor $p(X_t|\mathbf{X}_{0:t-1})$ corresponds to a Gaussian distribution with constant variance σ^2 (which we consider a hyperparameter) and a mean given by the **output of an RNN** with input X_{t-1}. The RNN state $\mathbf{h}_{t-1}$ therefore contains the information of the signal up to time $t-2$, that is, $\mathbf{X}_{0:t-2}$:

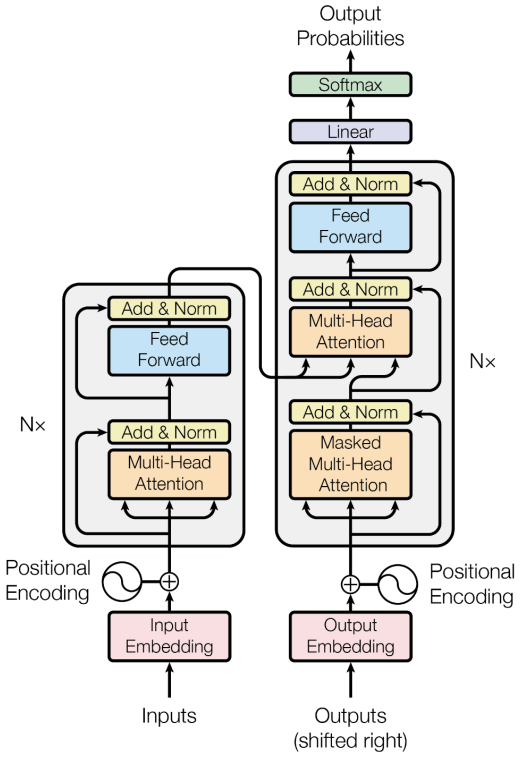
$$p(X_t|X_{t-1}, \mathbf{X}_{0:t-2}) = \mathcal{N}\left(f_{RNN}(X_{t-1}, \mathbf{h}_{t-1}), \sigma^2 + g_{RNN}(X_{t-1})\right)$$

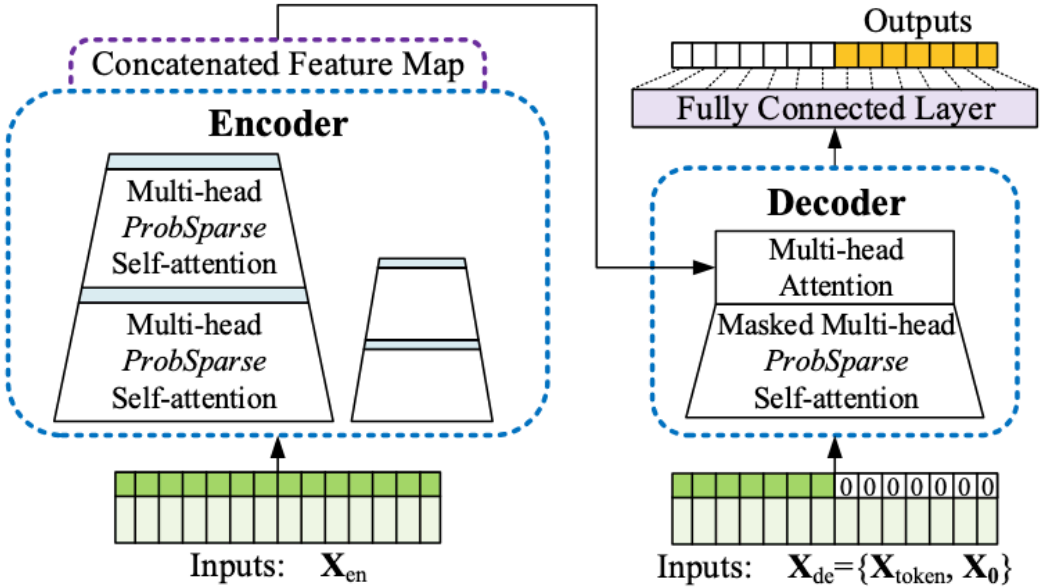
Important: during training, we feed the RNN with the **true** values of the signal, that is, at each time step $\mathbf{h}_{t-1}$ is computed using the real values of the signal. This technique is known as **teacher forcing**.

Generative Transformers

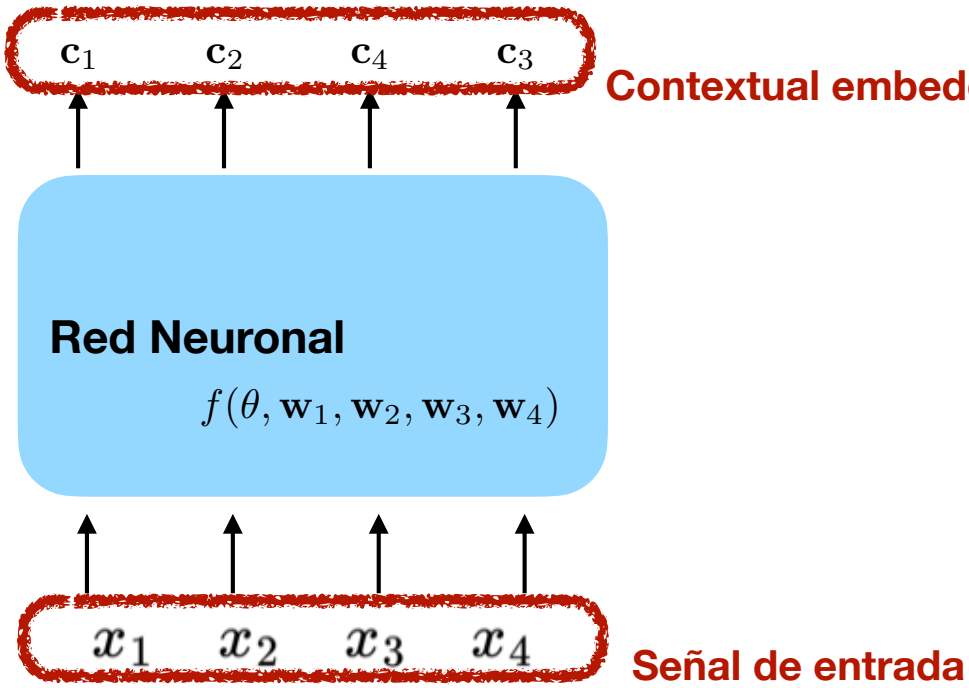
Modelos autoregresivos con transformers

- Estado del arte en modelado de series temporales
- En esencia, reemplazamos la RNN por una arquitectura de tipo *transformer*
- Totalmente paralelizable a través del mecanismo *self-attention*
- Utilización de máscaras para forzar causalidad



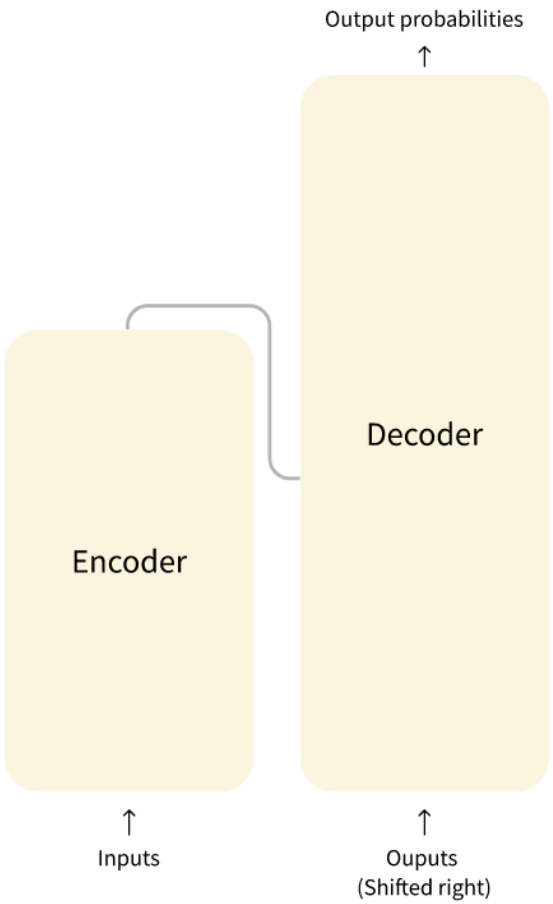


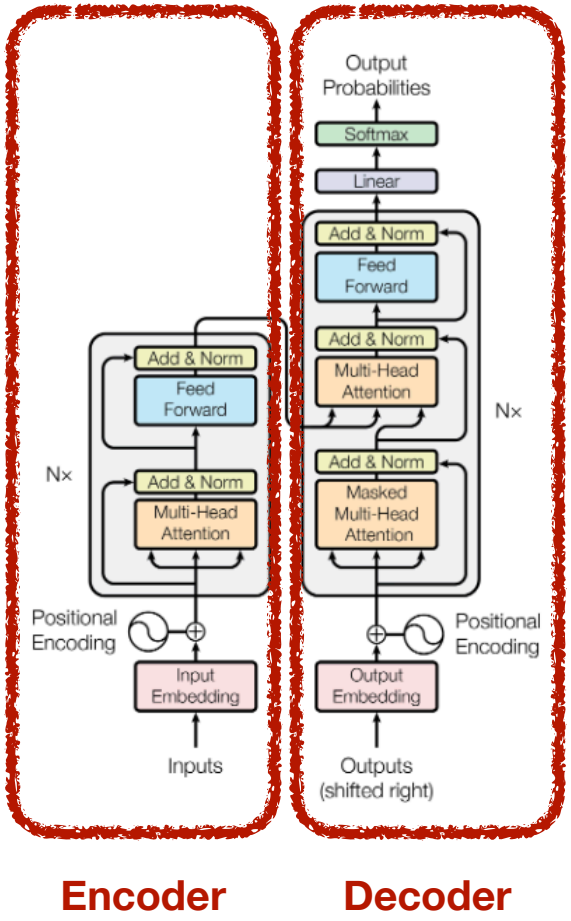
Transformers



- Feed-forward
- **Entrada**: vectores que representan la señal en cada instante
- **Salida:** *embedding contextuales,* tantos como longitud de la señal de entrada

Transformers

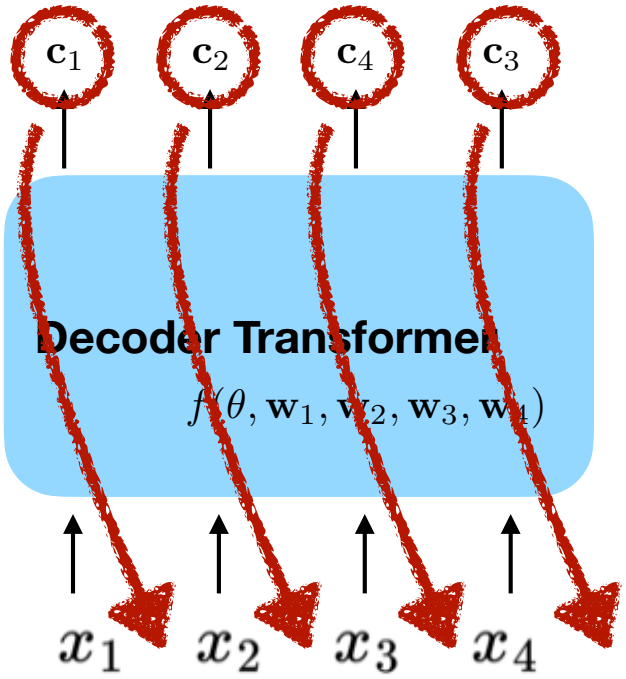




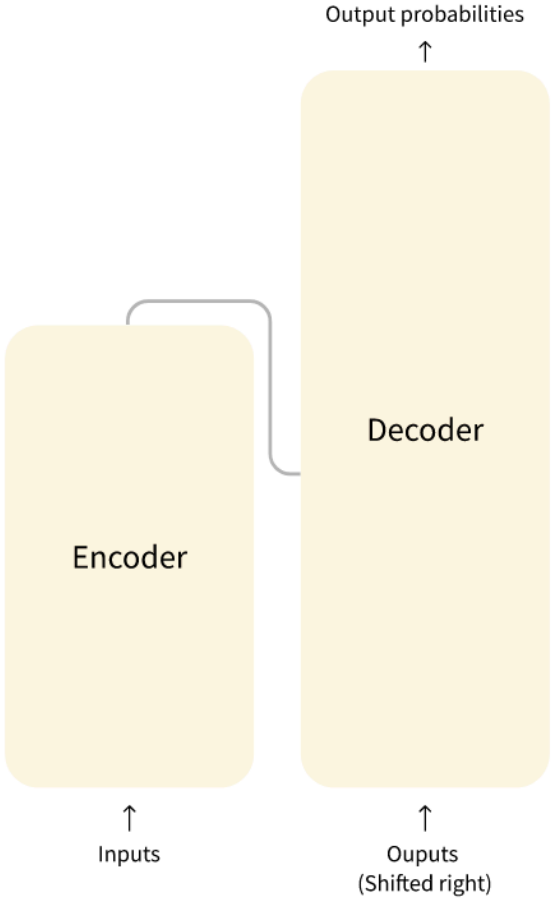
Cada uno de los bloques puede usarse independientemente, dependiendo de la tarea:

- Solo **encoder**: tareas de **clasificación** o **imputación no causal de valores perdidos**
- Sólo **decoder**: adecuado para **generación** (forecasting)
- **Encoder-decoder**: adecuado para tareas de **generación condicional**

Modelos AR series temporales con transformers



- Utilizamos cada embedding contextual para predecir el siguiente valor de la señal
- El encoder procesa la parte “observada” de la señal



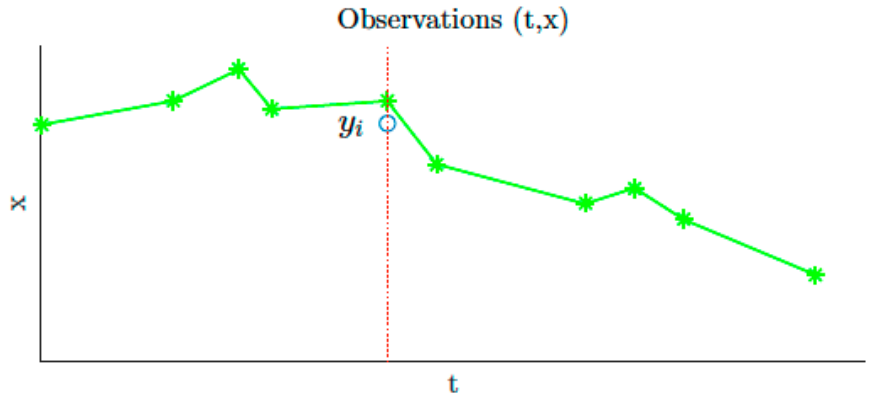
Self-attention: motivación

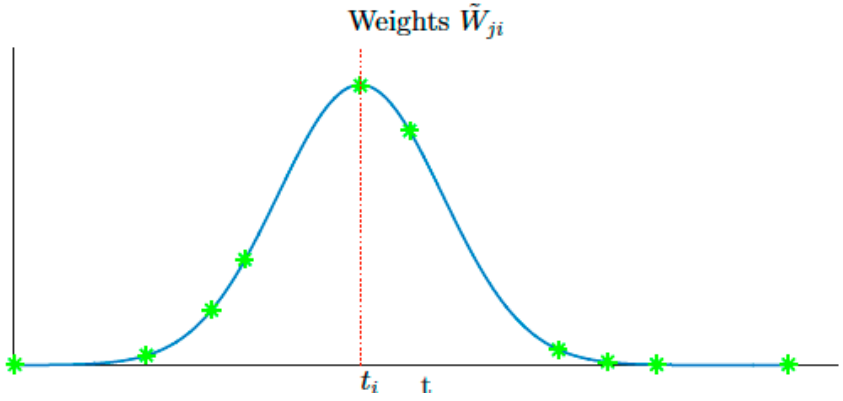
Pensemos en el suavizado (smoothing) de una señal

$$y_i = \sum_j w_{ji} x_j.$$

$$\tilde{w}_{ji} = \mathcal{N}(t_j; t_i, 1).$$

$$w_{ji} = \frac{\tilde{w}_{ji}}{\sum_r \tilde{w}_{ri}}$$





Self-attention (2)

Si lo que tenemos es una serie temporal multidimensional, podemos el **producto interno (~ similitud coseno)** para obtener los pesos del suavizado …

$$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5 \quad x_6 \quad x_7 \quad x_8 \quad x_9$$

$$y_9 = \sum_i w_i x_i = w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9$$

$$z_1 = x_1^\top x_9, z_2 = x_2^\top x_9, \ldots$$
$$[w_1, w_2, \ldots, w_9] = \mathrm{softmax}\left(z_1, z_2, \ldots, z_9\right)$$

Self-attention (3)

Esta operación la podemos generalizar, multiplicando los vectores por una matriz de parámetros …

$$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5 \quad x_6 \quad x_7 \quad x_8 \quad x_9$$

$$\begin{aligned}
\text{Query: } & q_9 = W_Q x_9, \\
\text{Keys: } & k_1 = W_K x_1, k_2 = W_K x_2, \ldots, k_9 = W_K x_9 \\
\text{Weights: } & \begin{cases} z_1 = k_1^\top q_9, z_2 = k_2^\top q_9, \ldots, z_9 = k_9^\top q_9 \\ [w_1, w_2, \ldots, w_9] = \text{softmax}\left(z_1, z_2, \ldots, z_9\right) \end{cases}
\end{aligned}$$

$$y_9 = \sum_i w_i x_i = w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9$$

Self-attention (4)

Self-attention: vector notation

- For all $i \in \{1, 2, \ldots, n\}$:

$$\begin{aligned} \text{Query:} \quad & q_i = W_Q x_i, \\ \text{Keys:} \quad & k_i = W_K x_i, \\ \text{Values:} \quad & v_i = W_V x_i. \end{aligned}$$

- For all $i, j \in \{1, 2, \ldots, n\}$:

$$Z_{ji} = k_j^T q_i / \sqrt{d}.$$

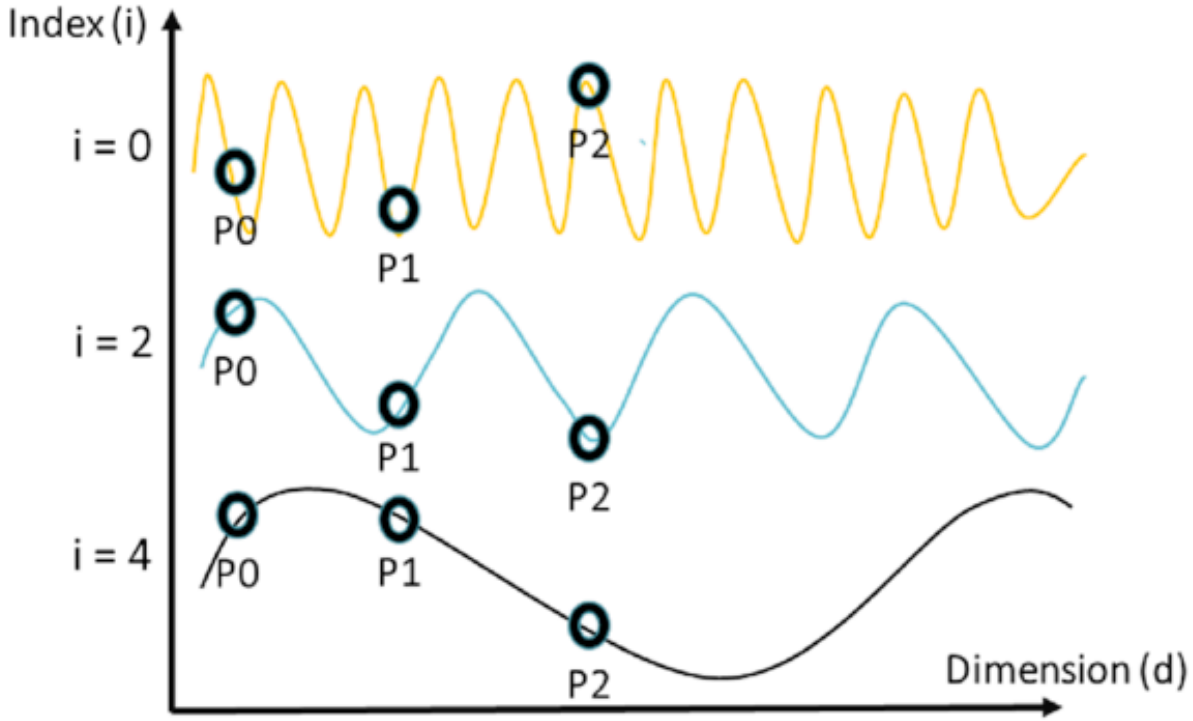
- For all $i \in \{1, 2, \ldots, n\}$:

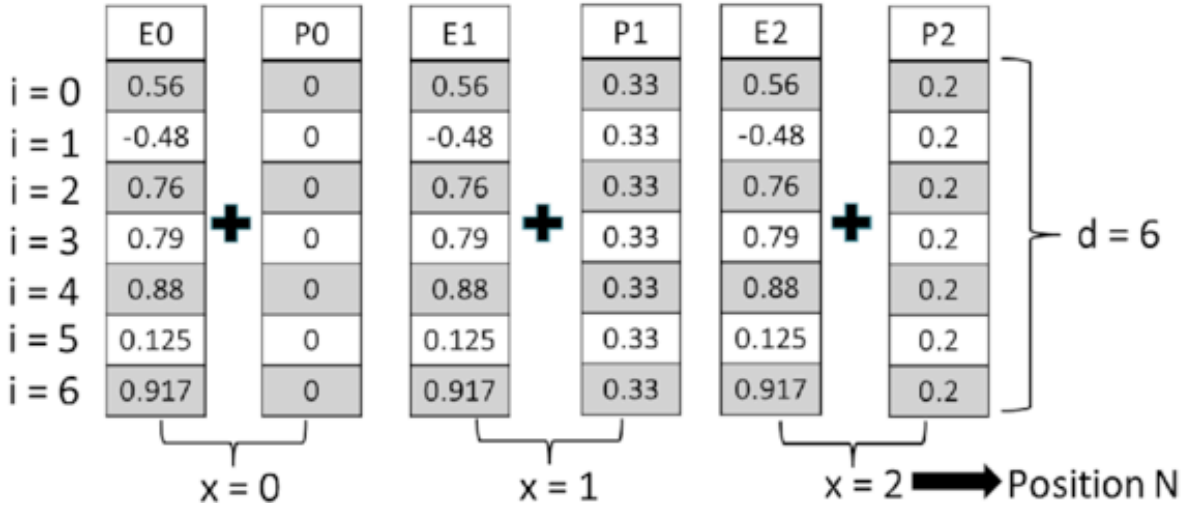
$$\text{Weights:} \quad [W_{1i}, W_{2i}, \ldots, W_{ni}] = \text{softmax}(Z_{1i}, Z_{2i}, \ldots, Z_{ni})$$

$$\text{New embedding:} \quad y_i = \sum_{j=1}^{n} v_j W_{ji}.$$

- El orden no importa! Se hace en paralelo de forma matricial!
- Para tener en cuenta el orden de la señal, concatenamos el *positional encoding*

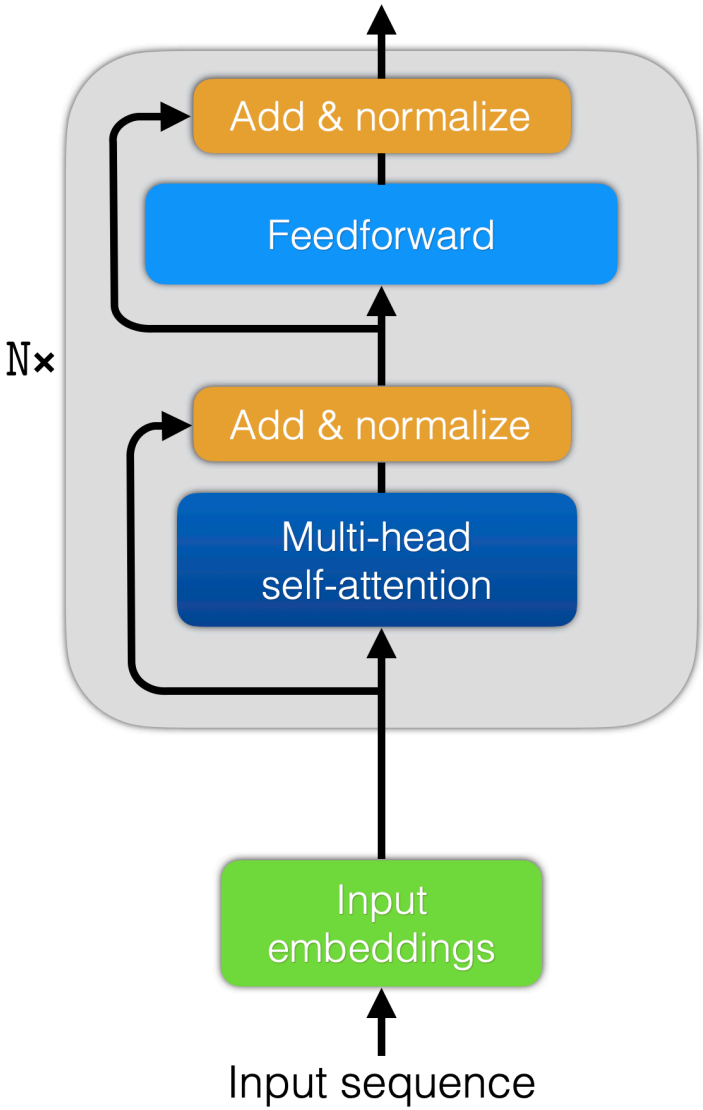
Positional encoding





$$x_i = Et_i \quad + \quad Pp_i$$

Transformer Encoder



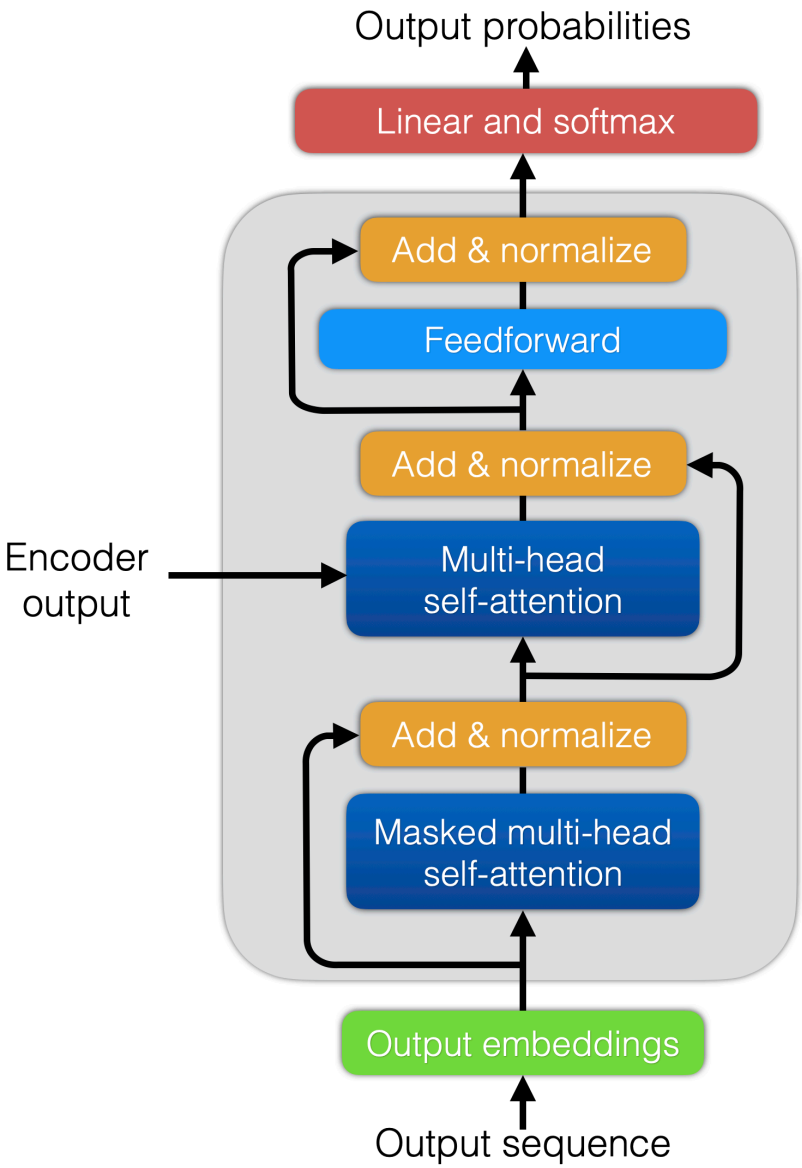
Capa residual + Normalización

Capa densa por embedding contextual

Capa residual + Normalización

Misma operación de self-attention, pero promediada entre varias inicializaciones de las matrices de parámetros (*heads)*

Transformer Decoder

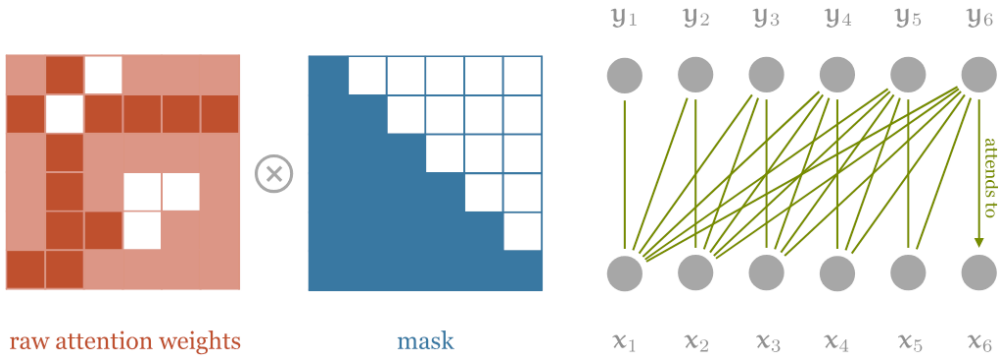


Predicción del siguiente valor de la señal

N×

Self attention en el que los pesos de atención se calculan usando la secuencia del encoder

Self attention **causal**



Informer: un ejemplo representativo

The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)

Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting

